



Capital Bikeshare & Building Footprint Segmentation Research Summary

Hourly Capital Bikeshare demand forecasting and Inria building-footprint segmentation in one research summary.

Prepared by: **Ibuchi Precious Basil**

Research Summary



Two parts

- 1 Demand forecasting
- 2 Building footprints segmentation



Capital Bikeshare Demand Analysis & Forecasting

What drives hourly bike rentals in Washington, D.C., and how well those rentals can be predicted an hour at a time.



Hourly demand

17,379 records · 2011–2012
Washington, D.C.



Contents

03

01

Executive Summary

The headline numbers

02

Introduction & Objectives

What the project set out to do

03

Data & Methodology

Where the data came from and how it was handled

04

Data Wrangling & Quality

Cleaning steps and what the checks showed

05

Exploratory Analysis

Timing, weather, and the two rider groups

06

Regression Analysis

OLS setup, results, and diagnostics

07

Demand Forecasting

Split design, model comparison, and drivers

08

Conclusions

What holds up, and what does not

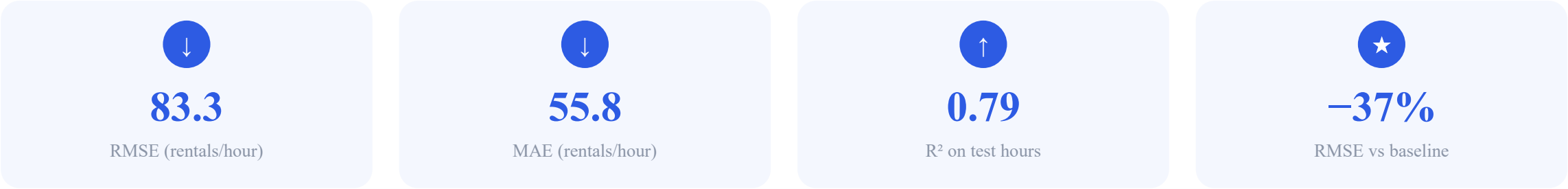


Executive Summary

04

Two years of hourly rentals (17,379 rows) were cleaned, explored, modelled with OLS, then forecast on days the models never trained on. Encoders and scalers were fit on the training days only, and casual and registered were kept out of the features.

BEST MODEL — RANDOM FOREST, HELD-OUT TEST HOURS



MODEL COMPARISON · TEST SET (DAY 21–END)

MODEL	RMSE	MAE	R ²
Hour-mean baseline	132.4	91.1	0.47
Linear regression	105.9	78.6	0.66
Random forest	83.3	55.8	0.79

KEY FINDINGS

- Demand peaks at 17:00–18:00 and again near 08:00
- Warmer hours are busier; humid hours are quieter
- Temperature effect bends, so temp² earned its place



Introduction & Objectives

05

Hourly rental volume moves with the clock and the weather. The work here measures those patterns, checks what shape they take, and then tests how much of an hour's demand can be predicted from what is already known about that hour.

WHAT THE PROJECT COVERS

- **Patterns.** How demand varies by hour, weekday, season, and weather, and where casual and registered riders differ.
- **Regression.** An OLS model linking hourly count to calendar and weather, with collinearity and non-linearity checked first.
- **Forecast.** Predicting total hourly rentals, trained and scored on separate days.

SCOPE SET UP FRONT

- Target is total hourly count, not rider groups
- Features come from the same hour only
- Train on days 1–20, test on day 21 onward
- casual and registered are never predictors

HOW THE SECTIONS CONNECT

- **EDA sets the story.** The hour and weather patterns it exposes are what the models are later expected to reproduce.
- **Regression tests the shape.** It showed the temperature effect is not a straight line, which is why temp^2 became a feature.
- **Forecasting is the check.** Held-out days show whether the patterns carry beyond the rows used for training.



Data & Methodology

06

The notebook runs as one forward pipeline, from download to forecast. The rules about what the model may see were settled before any model was fit.

DATA SOURCE

ITEM	DETAIL
Source	UCI Machine Learning Repository
System	Capital Bikeshare, Washington, D.C.
Grain	Hourly (hour.csv), 17,379 rows
Period	2011–2012 (two full years)
Target	count = casual + registered

RULES SET BEFORE MODELLING

- casual and registered stay out — they are the target
- Nothing from a later hour or a later day
- Encoders and scalers fit on training rows only
- dayofmonth splits the data, it is not a feature

PIPELINE



Tools: pandas and NumPy for wrangling, seaborn and matplotlib for the charts, statsmodels for OLS, and scikit-learn for the forecasting pipeline.



Data Wrangling & Quality

The raw hourly dataset was turned into a clean analysis table: a proper time index, clearer column names, and a set of checks run before anything was modelled.

CLEANING STEPS

- **Time index.** Built one sorted datetime column from the date and hour fields.
- **Naming.** Renamed the short source names (hr → hour, cnt → count, hum → humidity).
- **Validation.** Confirmed count = casual + registered on every row.
- **Range checks.** Checked the coded fields stay inside their documented ranges.
- **Split helper.** Added dayofmonth, used only for the split later on.

WHAT THE CHECKS SHOWED

- No missing values and no duplicate hours
- humidity = 0 in 22 hours — noted, left as recorded
- windspeed = 0 in 2,180 hours — calm hours, not errors
- No hour has zero rentals, so the target needs no special case

OUTPUT

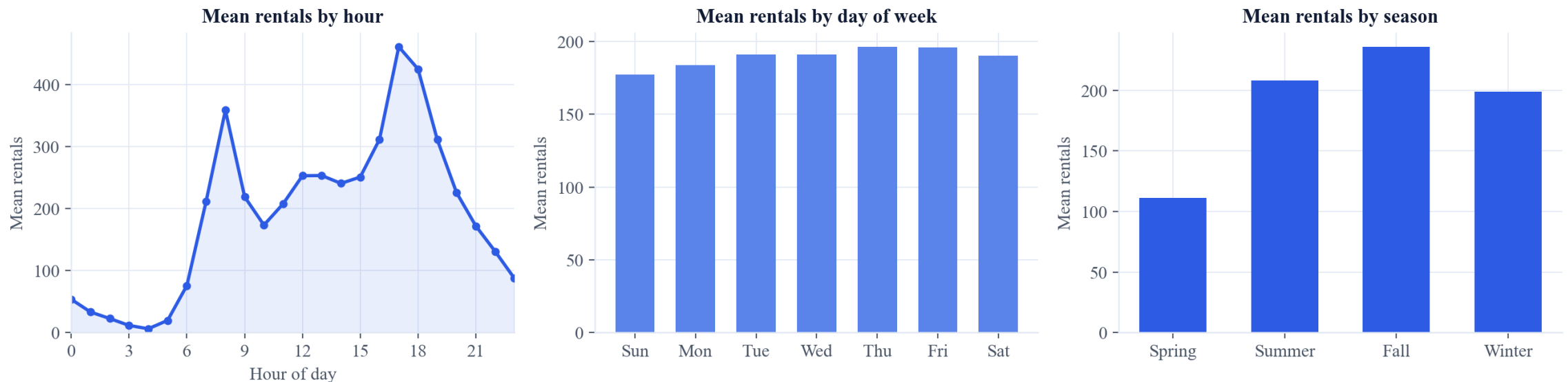
- **Cleaned table.** data/processed/bike_hourly_clean.csv, still holding all 17,379 hours.
- **Dropped here.** instant and the duplicate date field. atemp and workingday come out later, at the modelling stage.
- **Nothing filled in.** No imputing and no resampling, so the row count matches the source file.



Exploratory Analysis — Timing

08

Demand is right-skewed. The average hour sees about 190 rentals, but the busiest reaches 977, so a typical hour is quieter than the average suggests. Time of day is the strongest pattern here.



WHAT THE CHARTS SHOW

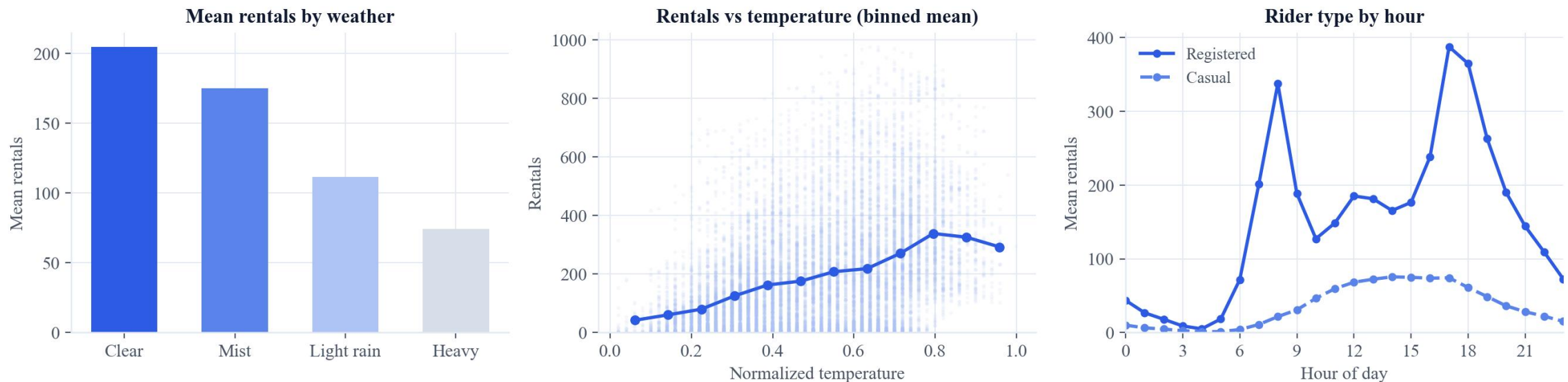
- **Hour of day.** Rentals peak in the late afternoon, around 17:00–18:00, and again near the 08:00 commute. Overnight demand is very low.
- **Weekday and season.** Working days sit only slightly above weekends, so the day of week matters far less than the hour. Fall and summer are the strongest seasons; spring is the weakest.



Exploratory Analysis — Weather & Riders

09

Weather moves the level of demand up or down, and the two rider groups do not behave the same way. Registered riders carry most of the volume: about 154 rentals an hour against 36 casual.



WHAT THE CHARTS SHOW

- **Weather and temperature.** Clear hours have the highest mean demand, and it drops as the weather worsens. Rentals climb with temperature then level off near the top of the range — the reason for testing a quadratic term. Heavy weather is rare in this data.
- **Rider type.** Registered demand carries the commute peaks. Casual demand is one broad mid-afternoon rise and runs higher on weekends and holidays.



Regression Analysis (OLS)

10

Ordinary Least Squares (OLS) was used to relate hourly count to the calendar and weather fields. The coded fields go in as categoricals, so the uneven time patterns from the EDA are picked up without forcing a straight line through them.

SPECIFICATION

- **Categoricals.** season, year, month, hour, holiday, weekday, weather.
- **Continuous.** temp, humidity, windspeed, plus temp² in the quadratic version.
- **Dropped atemp.** It carries nearly the same information as temp, so it added collinearity and nothing else.
- **Dropped workingday.** weekday and holiday already determine it. Leaving it out keeps the design matrix identifiable.

READING THE COEFFICIENTS

- **Temperature.** temp is positive and temp² is significant, so demand rises with warmth and then eases off.
- **Humidity and windspeed.** Both go with lower demand once the calendar effects are held fixed.
- **Weather.** Mist and light rain sit below clear conditions. Heavy weather is too rare here for a stable estimate.

MODEL SELECTION

TEMPERATURE TERM	ADJ. R ²	AIC
Linear temp	0.6851	210,053
Linear + temp²	0.6852	210,048

The quadratic version had the lower AIC, so temp + temp² was carried forward. It reaches $R^2 \approx 0.69$, or about two-thirds of the variation in hourly count.

WHERE TO BE CAREFUL

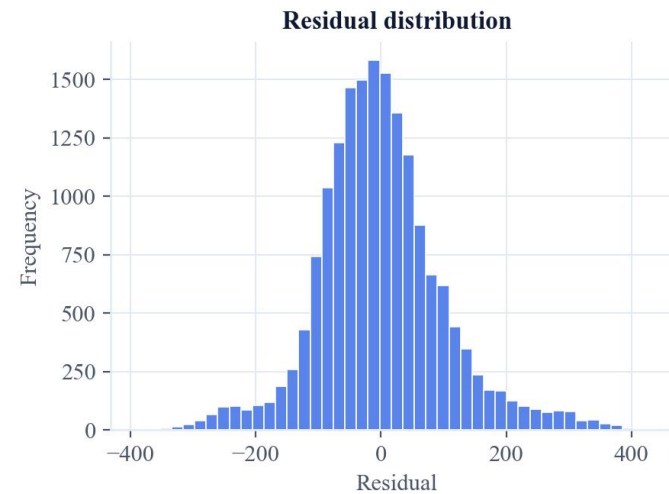
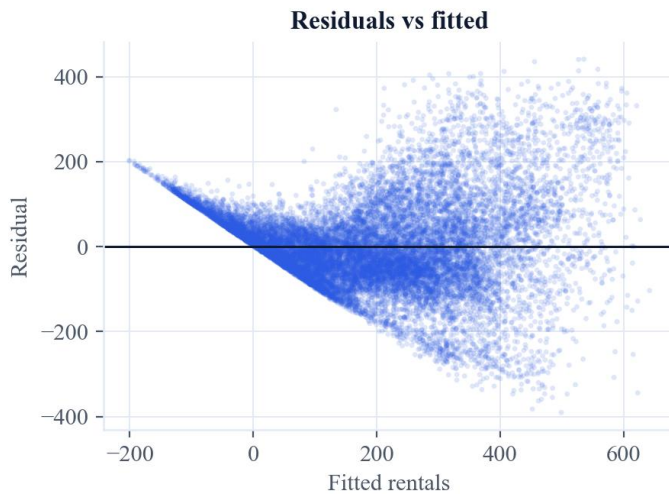
- A third of the variation is still unexplained
- count is skewed, so OLS is a baseline, not a count model
- These are associations, not proven causes



Regression Diagnostics

11

Fitted values are the rentals the model predicts for an hour. Residuals are what is left over, actual minus fitted. Together they show whether the errors behave well enough to trust the model.



WHAT EACH PLOT ANSWERS

- Left: are the errors even across quiet and busy hours?
- Right: are the errors centred, or leaning one way?
- Residuals sit roughly on zero, so there is no overall bias
- The spread is wide and grows at higher fitted values

READING

- **Good enough as a baseline.** Residuals stay centred on zero across the fitted range, so the model is not consistently guessing high or low.
- **The known weakness.** The widening spread and the right tail are what count data usually looks like. That is why a non-linear model was tried for the forecast.



Forecasting Design

12

The forecast only uses what is known about the hour being predicted. The split, the feature list, and the order of the preprocessing were all chosen with that in mind.

SPLIT

TRAIN · days 1–20

11,460 hours

TEST · day 21 to month end

5,919 hours

LEFT OUT ON PURPOSE

- casual, registered — they make up count
- atemp — collinear with temp
- workingday — weekday and holiday cover it
- dayofmonth — splits the data, nothing more

FEATURES AND PREPARATION

GROUP	FIELDS	TREATMENT
Calendar	hour, weekday, month, year, season, holiday	One-hot, drop first
Weather	weather (code), temp, humidity, windspeed	One-hot / standardised
Engineered	temp_sq (temp squared)	Standardised

The ColumnTransformer was fit on the training rows, then the same transform was applied to both sets. Nothing from the test days shaped the encoding levels or the scaling statistics.



Forecasting Results

13

Three models were scored on the same held-out hours. The hour-mean baseline predicts each hour's training average and nothing else, so it sets the floor. Random forest came out best on all three metrics.

TEST-SET PERFORMANCE

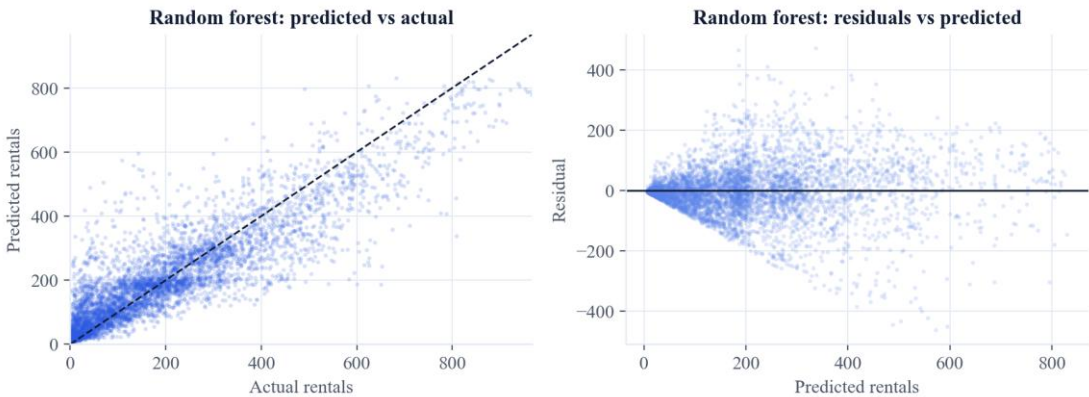
MODEL	RMSE	MAE	R ²
Hour-mean baseline	132.4	91.1	0.47
Linear regression	105.9	78.6	0.66
Random forest	83.3	55.8	0.79

Random forest cuts the baseline RMSE by 37% and linear regression's by 21%. The two differ because the effects do not simply add up.

READING THE DIAGNOSTICS

- **Where it holds.** Predictions follow actual rentals closely through the low and middle range, and the residuals stay centred on zero.
- **Where it slips.** The busiest hours are still under-predicted, so the forecast reads low exactly at the peaks.
- **Worth noting.** Every figure on this slide comes from day 21 onward — hours no model saw while training.

RANDOM FOREST ON THE TEST HOURS

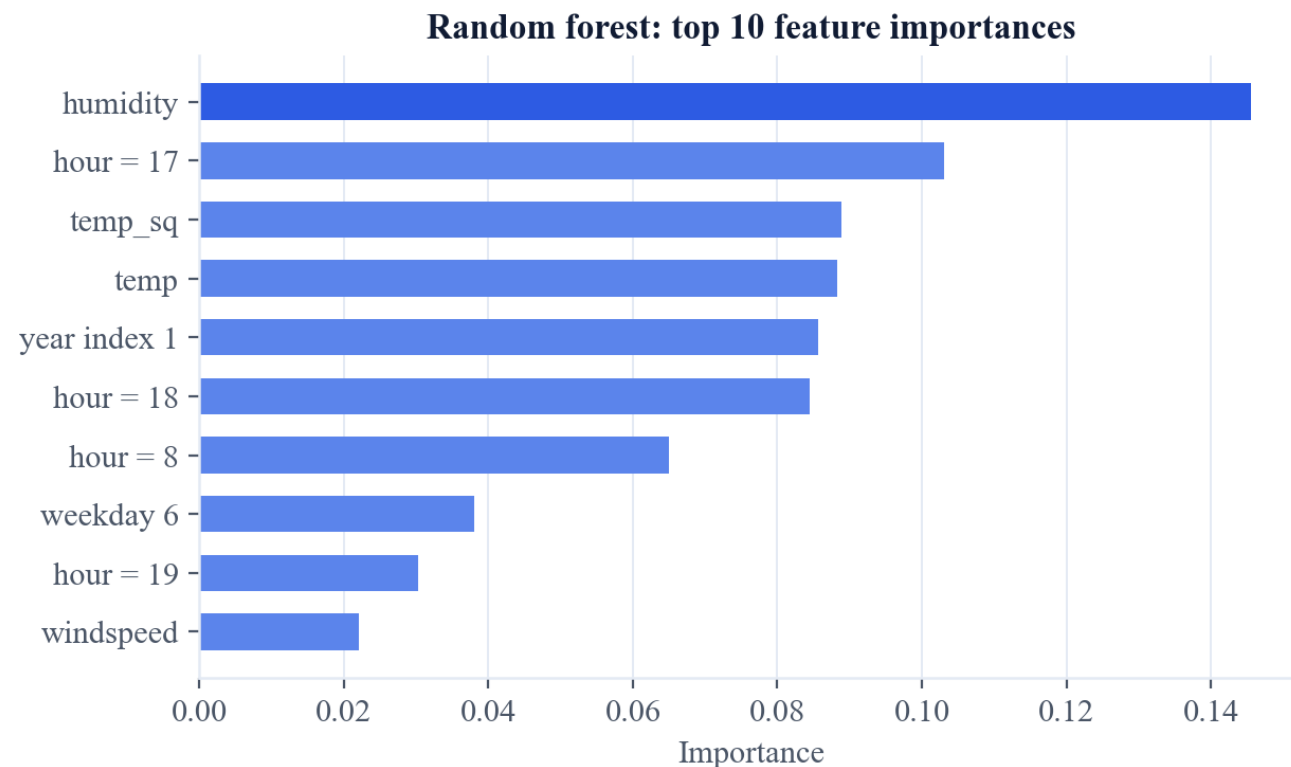




Demand Drivers

14

Feature importance shows which encoded inputs the random forest leaned on. It ranks how much each one helped accuracy — it does not say which direction the effect runs.



TOP OF THE LIST

- humidity, 0.146 — the highest single feature
- hour = 17, 18 and 8 — the commute peaks
- temp with temp_sq, 0.177 between them
- year, 0.086 — 2011 and 2012 sit at different levels

HOW TO READ IT

- Hour is spread across separate dummy bars, so timing counts for more than any one row shows.
- temp and temp_sq are one temperature effect and belong together.
- This matches the EDA: hour and temperature lead.



Conclusions

15

Hourly demand comes down to when it is and what the weather is doing, and that is enough to forecast it on days the models never trained on.

WHAT THE WORK SHOWS

- **Drivers.** Time of day leads, with the commute peaks doing most of the work; weather moves the level up or down. Casual and registered riders follow different patterns.
- **Shape.** The temperature effect is not linear. Quadratic temp fit better than temp alone, giving $R^2 \approx 0.69$ at a lower AIC.
- **Forecast.** Same-hour calendar and weather alone reach R^2 0.79 and RMSE 83.3 with the random forest, well past the hour-mean baseline.
- **Method.** The days 1–20 / 21–end split with train-only encoding and scaling is what makes those numbers believable.

LIMITATIONS

- The busiest hours are still under-predicted
- Two years of one city, nothing past 2012
- Weather here is recorded, not forecast
- Associations, not proven causes

WHAT COULD FOLLOW

- **A count model.** Poisson or negative binomial would suit a skewed count target better than OLS does.
- **Tuning.** A small hyperparameter search inside the training window, validated with a time-series split.
- **Split by rider type.** Casual and registered behave differently, so modelling them separately is worth testing.



Building Footprint Segmentation

How well a U-Net can map building footprints on Inria aerial imagery, scored on a city it never trained on.



Vienna hold-out

64,980 patches · 10 epochs

Best IoU 0.627



Contents

17

01 Executive Summary

Headline Vienna scores

02 Introduction & Objectives

Task, scope, and how the sections connect

03 Data & Methodology

Inria source and the forward pipeline

04 Acquisition & Patching

Tiles on disk, then 256x256 windows

05 City Hold-out Split

Vienna eval; no neighbor-patch leakage

06 Model & Loss

U-Net, BCE+Dice, IoU and Dice

07 Training & Results

M1 run, curves, and the epoch log

08 Visual Error Analysis

Vienna panels across an IoU ladder

09 Conclusions

What holds up, and what could follow



Executive Summary

Inria aerial tiles were patched to 256x256, trained on four cities, and scored on Vienna only. A from-scratch U-Net with BCE-with-logits + soft Dice produced usable footprints; the best checkpoint is epoch 8 of 10 on Apple M1 (MPS).

BEST VIENNA SEGMENTATION · U-NET, EPOCH 8

**0.627**

Eval IoU

**0.731**

Eval Dice

**0.628**

Eval loss

**epochs**

M1 MPS train

KEY TAKEAWAYS

- **Hold-out.** Vienna-only eval blocks neighbor-patch and city bleed into the score.
- **Loss.** BCE + Dice handles class imbalance better than pixel accuracy alone.
- **Peak.** IoU and Dice peak at epoch 8; later epochs did not beat that checkpoint.
- **Failures.** Small or unclear buildings are still the weak cases on the IoU ladder.



Introduction & Objectives

Binary building vs background on Inria aerial imagery, scored on Vienna.

WHAT THE PROJECT COVERS

- **Data.** Inria tiles patched to 256×256, with image and mask crops kept aligned.
- **Split.** Vienna held out by city so neighboring patches cannot leak into the score.
- **Model.** From-scratch U-Net with BCE-with-logits + soft Dice, then IoU/Dice at 0.5.

SCOPE SET UP FRONT

- Binary footprint masks only
- Train cities exclude Vienna
- Hybrid CNN–ViT left out of scope
- Best weights chosen by Vienna IoU

HOW THE SECTIONS CONNECT

- **Patching makes training feasible.** Full 5000×5000 tiles do not fit the model in one pass on this machine.
- **City hold-out makes the score honest.** Random patch splits would overstate transfer by sharing neighbors.
- **Visuals explain the number.** The IoU ladder shows where footprints hold and where they fail.



Data & Methodology

One forward pipeline from Inria tiles to Vienna scores.

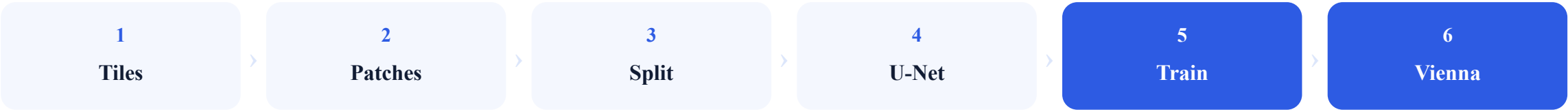
DATA SOURCE

ITEM	DETAIL
Source	Inria Aerial Image Labeling
Task	Building vs background
Tile size	5000×5000 RGB @ 0.3 m
Cities	Austin, Chicago, Kitsap, Tyrol, Vienna
Labels	255 = building, 0 = background

RULES SET BEFORE MODELLING

- Split by city, never by random patch
- Vienna is evaluation only
- Same crop on image and gt mask
- Best checkpoint chosen by eval IoU

PIPELINE



Tools: PyTorch (MPS on M1 for training), torchvision-style transforms, matplotlib for curves and prediction panels, PIL for patch I/O.



Acquisition & Inspection

21

Train tiles arrive paired with gt masks; buildings are a small share of pixels.

WHAT WAS LOADED

- **Train.** 180 aerial tiles paired with 180 gt masks.
- **Test.** 180 aerial tiles; no public masks used here.
- **Shape.** Sample tile is 5000×5000 RGB.
- **Cities.** austin, chicago, kitsap, tyrol, vienna.

WHAT THE CHECKS SHOWED

- **Labels.** Mask values are 0 / 255 only.
- **Pairs.** Train image and gt filenames match.
- **Imbalance.** Background dominates; accuracy alone is weak.
- **Hard edges.** Footprints need sharp boundaries, not blur.



Patching

Full tiles are too large for one forward pass; aligned 256 windows keep resolution.

SETTINGS

- **Size.** 256×256 with stride 256 (non-overlapping).
- **Pairs.** Same crop on image and gt mask.
- **Edges.** Leftover strips smaller than 256 px dropped.
- **Output.** 64,980 image/mask PNG pairs under processed/.

TRADE-OFFS

- **Gain.** Fits training memory; many examples per tile.
- **Cost.** Less whole-tile context per sample.
- **Borders.** Buildings on patch edges can be split.
- **Fix later.** City hold-out still blocks neighbor leakage.



City Hold-out Split

23

Split by city, not by random patch, so Vienna scores are not inflated by spatial bleed.

TRAIN · Austin, Chicago, Kitsap, Tyrol

51,984 patches · 144 tiles across 4 cities

EVAL · Vienna only

12,996 patches · 36 Vienna tiles

WHY NOT A RANDOM PATCH SPLIT

- **Neighbor leak.** Adjacent crops from the same tile would land in both sets.
- **City bleed.** Same-city look and layout would inflate transfer scores.
- **Check.** No city overlap and no tile overlap between train and eval.



Transforms & DataLoaders

Train patches get light geometry; eval stays fixed so Vienna scores stay comparable.

DATASET

- **Class.** PatchSegDataset pairs each image with its mask.
- **Labels.** Building pixels mapped to class 1.
- **Tensors.** Images CHW float in $[0, 1]$; masks HxW ids.
- **Batch.** (B, 3, 256, 256) images, (B, 256, 256) masks.
- **B.** Batch size 4.

AUGMENTATION

- **Train.** Random flips and 90° rotations.
- **Joint.** Same transform on image and mask.
- **Eval.** Tensor convert only; no random ops.
- **Shuffle.** Train shuffles; eval does not.
- **Why.** Keeps labels aligned and eval stable.



Model — U-Net

25

Encoder–decoder with skips so every pixel gets a building logit at full patch size.

ARCHITECTURE

- **Shape.** Four encoder stages, bottleneck, four decoder stages.
- **Block.** DoubleConv: Conv-BN-ReLU $\times 2$.
- **Width.** Base $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ (~7.8M params).
- **Output.** One logit channel per pixel.
- **Infer.** Sigmoid, then threshold 0.5.

WHY U-NET HERE

- **Need.** Full HxW mask, not one patch score.
- **Skips.** Pass fine detail for building edges.
- **Size.** Light enough for local MPS training.
- **Check.** Forward pass keeps 256×256 spatial size.
- **Scope.** Hybrid CNN–ViT skipped; CNN covers the brief.



Loss & Metrics

Background dominates, so pixel accuracy alone is a weak guide.

COMBINED TRAIN LOSS

- **BCE.** Per-pixel building vs background on logits.
- **Dice.** Soft overlap on sigmoid probabilities.
- **Mix.** $L = \text{BCE-with-logits} + \text{Dice}$ (equal weight).
- **Why both.** BCE alone under-weights rare roofs; Dice alone can be noisy early.
- **Infer.** Hard mask via sigmoid + 0.5; no BCE needed.

REPORTED METRICS

- **IoU.** Intersection over union on the hard mask.
- **Dice.** Overlap score on the hard mask.
- **City.** Both scored on Vienna only.
- **Pick.** Checkpoint saved when eval IoU improves.
- **Skip.** Pixel accuracy stays secondary.



Training Setup

Full Inria training is impractical on CPU here; the run used Apple M1 MPS.

RUN SETTINGS

- **Machine.** MacBook Pro M1, PyTorch MPS.
- **Optim.** Adam, lr 0.001, batch 4.
- **Epochs.** 10 full passes over train patches.
- **Eval.** Vienna every epoch (loss, IoU, Dice).
- **Save.** Best weights by eval IoU → unet_best.pth.

THIS NOTEBOOK MACHINE

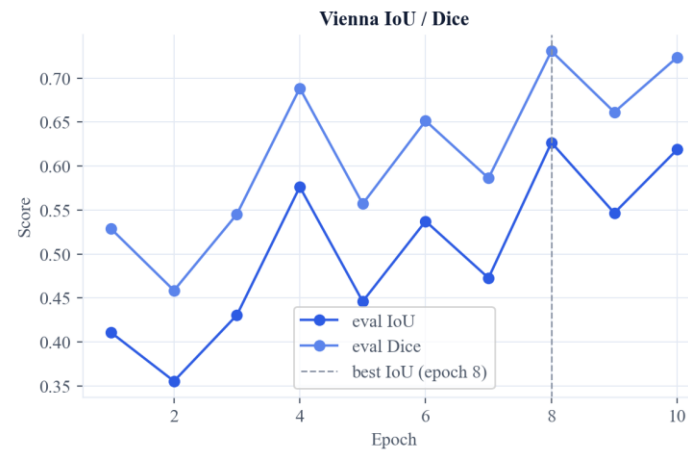
- **Loop.** Train cell kept, gated by RUN_TRAINING=False.
- **Load.** Checkpoint restored with map_location.
- **Why.** CPU would be roughly hours per epoch.
- **Continue.** Curves, Vienna scores, and visuals use the saved run.
- **Note.** History for plots covers all 10 epochs.



Training Results

28

Train loss fell across epochs; Vienna IoU bounced and peaked at epoch 8.



BEST RUN

- **Best.** Epoch 8 of 10.
- **IoU.** 0.6267 on Vienna.
- **Dice.** 0.7307 on Vienna.
- **Eval loss.** 0.6283 at best epoch.
- **Trend.** Train loss 0.78 → 0.38.
- **Close.** Epoch 10 near best; did not beat it.



Epoch Log

Vienna metrics from the M1 training run. Best checkpoint marked at epoch 8.

EPOCH	TRAIN LOSS	EVAL LOSS	IOU	DICE
1	0.7814	1.0067	0.4107	0.5286
2	0.5735	1.2115	0.3554	0.4585
3	0.5080	0.9596	0.4305	0.5452
4	0.4740	0.7326	0.5763	0.6885
5	0.4450	0.9811	0.4460	0.5570
6	0.4300	0.7998	0.5373	0.6514
7	0.4156	0.9348	0.4727	0.5862
8 ★	0.4028	0.6283	0.6267	0.7307
9	0.3956	0.8201	0.5466	0.6613
10	0.3839	0.6473	0.6188	0.7237

Dip at epoch 2, another soft stretch around epochs 5–7, then the IoU peak at epoch 8. Saved weights stay on that best checkpoint.



Visual Error Analysis

30

Vienna panels across an IoU ladder (buildings + background in each patch).



Reading the ladder

Near 1.0: almost perfect match to GT.

~0.75: clear separation; most footprint right.

~0.50: several buildings recovered; others missed.

~0.17: weaker, but some building pixels remain.

~0: all-background pred; no buildings found.



Conclusions

City hold-out, a compact U-Net, and BCE+Dice gave a usable Vienna footprint model and made the failure modes easy to see.

WHAT THE WORK SHOWS

- **Split.** Vienna-only evaluation blocks neighbor-patch and city bleed into the score.
- **Model.** From-scratch U-Net with BCE+Dice learns usable footprints under class imbalance.
- **Best score.** IoU 0.6267 / Dice 0.7307 at epoch 8; later epochs did not beat it.
- **Visuals.** High IoU panels match GT; mid IoU recovers some roofs; IoU ~0 stays all background.

WHAT COULD FOLLOW

- **Longer train.** More epochs or early-stop patience around the IoU peak.
- **Stronger encoder.** A wider backbone or pretrained encoder for harder roofs.
- **Small buildings.** Extra focus on thin or ambiguous footprints that fail at low IoU.

LIMITATIONS

- Small or unclear buildings still missed
- 10 epochs; no encoder search
- Patch borders can split footprints
- Hybrid CNN–ViT not attempted

Thank you

Happy to take questions.

Notebooks: [capital-bikeshare-demand-analysis.ipynb](#) · [building-footprint-segmentation.ipynb](#)

Ibuchi Precious Basil

